

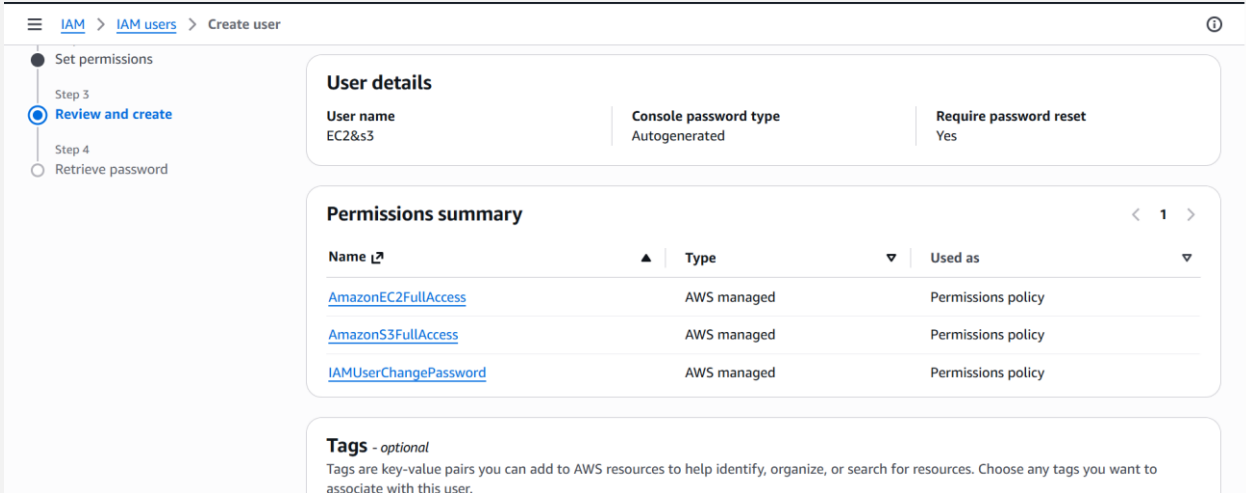
AWS IAM Tasks

1. Create an IAM User with EC2 Full Access + S3 Full Access

This task creates a single IAM user and attaches two AWS managed policies granting full access to EC2 and S3.

STEP 1 Create the IAM User

- Sign in to the AWS Management Console → open IAM.
- Left panel → Users → Create user.
- Enter username: Ec2-s3-admin.
- Enable console access and set a password (optional).

 **SCREENSHOT: IAM → Create user form**

The screenshot shows the 'Create user' form in the AWS IAM console. The breadcrumb trail is IAM > IAM users > Create user. The left sidebar shows a progress bar with four steps: Set permissions, Review and create (selected), Retrieve password, and Retrieve password. The main content area is divided into three sections: User details, Permissions summary, and Tags - optional.

User details

User name	Console password type	Require password reset
EC2&S3	Autogenerated	Yes

Permissions summary

Name	Type	Used as
AmazonEC2FullAccess	AWS managed	Permissions policy
AmazonS3FullAccess	AWS managed	Permissions policy
IAMUserChangePassword	AWS managed	Permissions policy

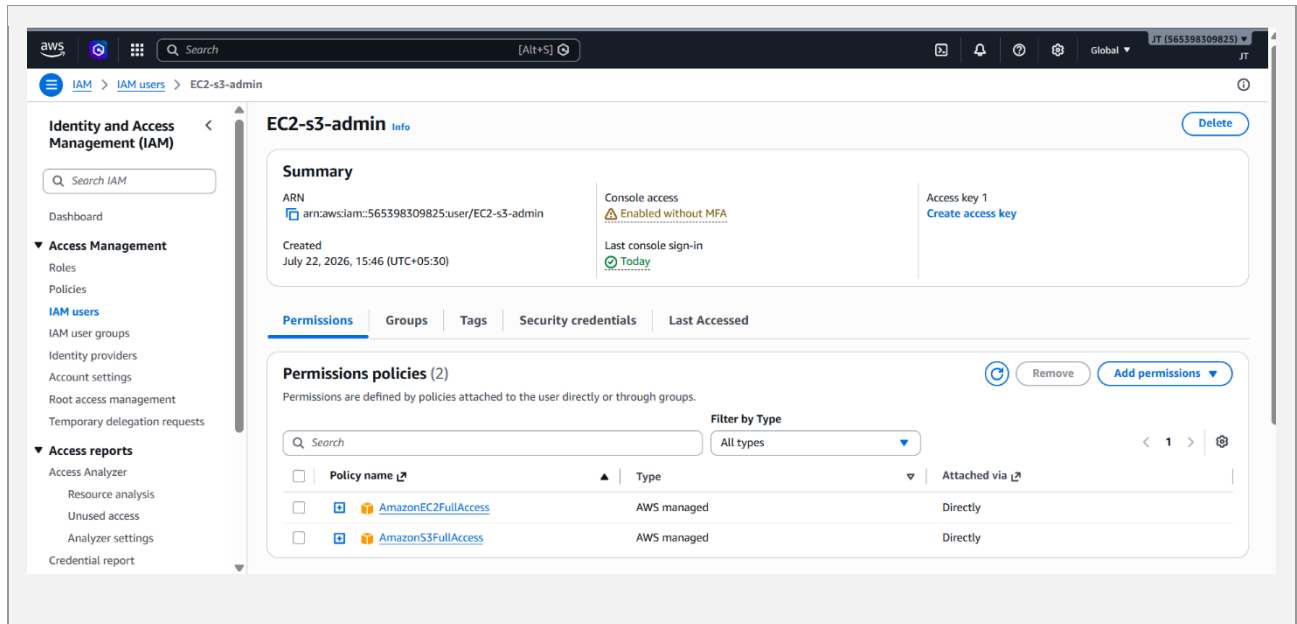
Tags - optional

Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.

STEP 2 Attach EC2 and S3 Full Access Policies

- On the "Set permissions" page, choose Attach policies directly.
- Search for and select: AmazonEC2FullAccess
- Search for and select: AmazonS3FullAccess
- Click Next → Review → Create user.

 **SCREENSHOT: Attach policies — AmazonEC2FullAccess and AmazonS3FullAccess selected**



AWS CLI equivalent:

```
# Create the user
aws iam create-user --user-name ec2-s3-admin

# Attach EC2 Full Access
aws iam attach-user-policy \
  --user-name ec2-s3-admin \
  --policy-arn arn:aws:iam::aws:policy/AmazonEC2FullAccess

# Attach S3 Full Access
aws iam attach-user-policy \
  --user-name ec2-s3-admin \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3FullAccess

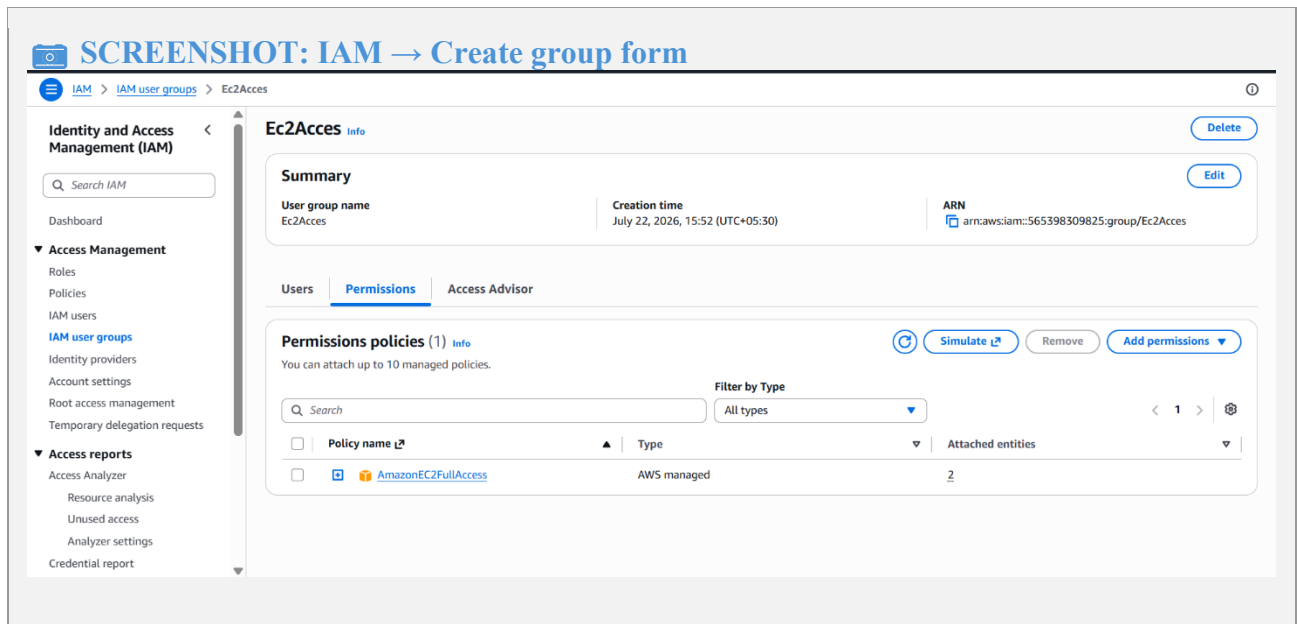
# (Optional) create access keys for programmatic access
aws iam create-access-key --user-name ec2-s3-admin
```

2. Create an IAM Group with EC2 Read-Only Access

This task creates an IAM group and attaches a read-only EC2 policy so any member automatically inherits read-only EC2 permissions.

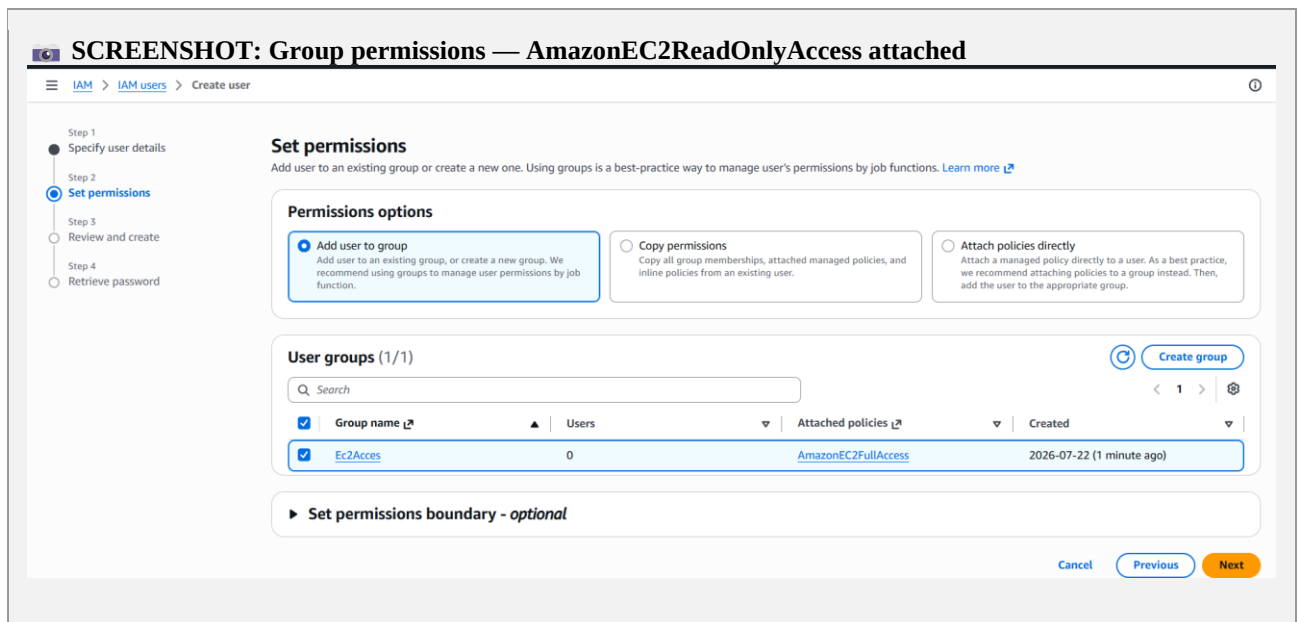
STEP 3 Create the Group

- IAM → User groups → Create group.
- Group name: ec2-readonly-group.



STEP 4 Attach the EC2 Read-Only Policy

- Under "Attach permissions policies", search for and select: AmazonEC2ReadOnlyAccess
- Click Create group.



AWS CLI equivalent:

```
# Create the group
```

```
aws iam create-group --group-name ec2-readonly-group

# Attach the EC2 Read-Only policy
aws iam attach-group-policy \
  --group-name ec2-readonly-group \
  --policy-arn arn:aws:iam::aws:policy/AmazonEC2ReadOnlyAccess
```

3. Create User "Devops" and Add to the Group from Task 2

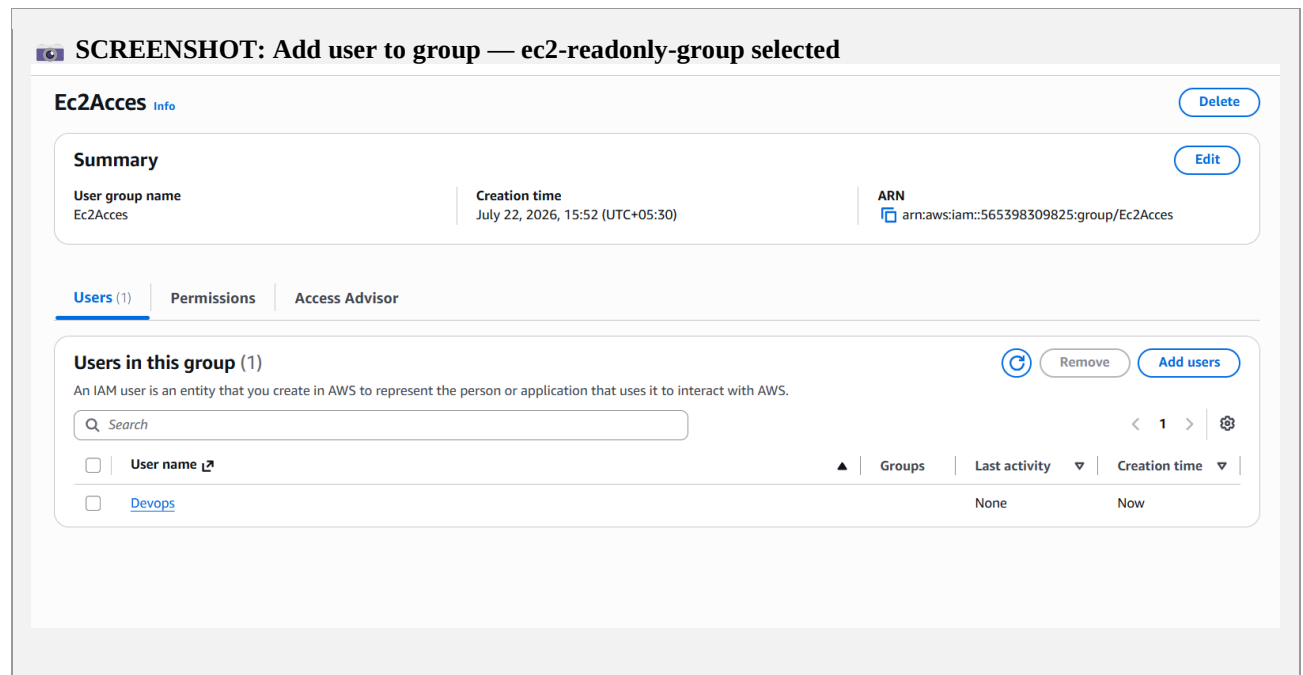
This task creates a new IAM user named Devops and places it in the ec2-readonly-group created in Task 2, so it inherits EC2 read-only permissions.

STEP 5 Create the User

- IAM → Users → Create user.
- Username: Devops.

STEP 6 Add the User to the Group

- On the "Set permissions" page, choose Add user to group.
- Select ec2-readonly-group.
- Click Next → Review → Create user.

The screenshot shows the AWS IAM console interface. At the top, a title bar reads "SCREENSHOT: Add user to group — ec2-readonly-group selected". Below this, the "Ec2Acces" group page is displayed. The "Summary" tab is active, showing the group name "Ec2Acces", creation time "July 22, 2026, 15:52 (UTC+05:30)", and ARN "arn:aws:iam::565398309825:group/Ec2Acces". The "Users" tab is selected, showing "Users in this group (1)". A table lists the user "Devops" with checkboxes for selection. The table has columns for "User name", "Groups", "Last activity", and "Creation time". The "Devops" user is listed with "None" for groups and "Now" for creation time.

SCREENSHOT: Add user to group — ec2-readonly-group selected

Ec2Acces [Info](#) [Delete](#)

Summary [Edit](#)

User group name: Ec2Acces | Creation time: July 22, 2026, 15:52 (UTC+05:30) | ARN: [arn:aws:iam::565398309825:group/Ec2Acces](#)

[Users \(1\)](#) | [Permissions](#) | [Access Advisor](#)

Users in this group (1) [Refresh](#) [Remove](#) [Add users](#)

An IAM user is an entity that you create in AWS to represent the person or application that uses it to interact with AWS.

☐	User name ↗	▲ Groups	Last activity ▼	Creation time ▼
☐	Devops	None		Now

AWS CLI equivalent:

```
# Create the Devops user
aws iam create-user --user-name Devops

# Add Devops to the group created in Task 2
aws iam add-user-to-group \
  --user-name Devops \
  --group-name ec2-readonly-group
```

NOTE: Devops now inherits AmazonEC2ReadOnlyAccess permissions automatically through group membership — no policy is attached directly to the user.

4. Bash Script to Create an IAM User with VPC Full Access

This task uses a bash script (calling the AWS CLI) to create an IAM user and grant it full VPC access.

STEP 7 Write the Script

Save the following as create_vpc_user.sh:

```
#!/bin/bash

# Script to create an IAM user with VPC Full Access

# Enter IAM username
read -p "Enter IAM username: " USERNAME

# Create IAM user
aws iam create-user --user-name "$USERNAME"

if [ $? -eq 0 ]; then
  echo "IAM user '$USERNAME' created successfully."
else
  echo "Failed to create IAM user."
  exit 1
fi

# Attach AmazonVPCFullAccess policy
aws iam attach-user-policy \
  --user-name "$USERNAME" \
  --policy-arn arn:aws:iam::aws:policy/AmazonVPCFullAccess

if [ $? -eq 0 ]; then
  echo "AmazonVPCFullAccess policy attached successfully."
else
  echo "Failed to attach policy."
  exit 1
fi
```

```
echo "=====
echo "IAM User: $USERNAME"
echo "Policy : AmazonVPCFullAccess"
echo "Status : Completed Successfully"
echo "=====
```

STEP 8 Run the Script

```
chmod +x create_vpc_user.sh
./create_vpc_user.sh
```

SCREENSHOT: Terminal output after running create_vpc_user.sh

```
DELL@KHADEER MINGW64 ~/Downloads
$ aws --version
aws-cli/1.45.48 Python/3.14.4 Windows/11 botocore/1.43.48

DELL@KHADEER MINGW64 ~/Downloads
$ aws configure
AWS Access Key ID [*****H5WR]: AKIAYHJC5RPASN4RB3HQ
AWS Secret Access Key [*****mNfp]: VmVNILP7oNZG9LVFSmFxhEAL4KU7DaSFqiSZ9g
Dw
Default region name [ap-south-2]: us-east-1
Default output format [None]: json

DELL@KHADEER MINGW64 ~/Downloads
$ vi create_iam_user.sh

DELL@KHADEER MINGW64 ~/Downloads
$ chmod +x create_iam_user.sh

DELL@KHADEER MINGW64 ~/Downloads
$ ./create_iam_user.sh
Enter IAM username: CLIuser
{
  "User": {
    "Path": "/",
    "UserName": "CLIuser",
    "UserId": "AIDAYHJC5RPAU552LEBPE",
    "Arn": "arn:aws:iam::565398309825:user/CLIuser",
    "CreateDate": "2026-07-22T10:38:12Z"
  }
}
IAM user 'CLIuser' created successfully.
AmazonVPCFullAccess policy attached successfully.
=====
IAM User: CLIuser
Policy : AmazonVPCFullAccess
Status : Completed Successfully
=====
```

CLluser
Info
Delete

Summary

ARN

arn:aws:iam::565398309825:user/CLluser

Created

July 22, 2026, 16:08 (UTC+05:30)

Console access

Disabled

Last console sign-in

-

Access key 1

Create access key

Permissions
Groups
Tags
Security credentials
Last Accessed

Permissions policies (1)

Remove

Add permissions

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type

All types

Policy name

Type

Attached via

☐
☐

AmazonVPCFullAccess

AWS managed

Directly

Permissions boundary (not set)

5. IAM Policy — EC2 Access for a Specific User, Restricted to Specific Regions

This task creates a custom IAM policy that allows EC2 actions only when the request is made in an approved AWS region, using the `aws:RequestedRegion` condition key.

STEP 9 Create the Policy JSON

Save as `ec2-region-restricted-policy.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowEC2OnlyInSpecificRegions",
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": ["us-east-1", "us-west-1"]
        }
      }
    },
    {
      "Sid": "DenyEC2OutsideSpecificRegions",
      "Effect": "Deny",
```

```

    "Action": "ec2:*",
    "Resource": "*",
    "Condition": {
        "StringNotEquals": {
            "aws:RequestedRegion": ["us-east-1", "us-west-2"]
        }
    }
}
]
}

```

STEP 10 Create and Attach the Policy

- Console: IAM → Policies → Create policy → JSON tab → paste the JSON above.
- Name it EC2-Region-Restricted-Policy → Create policy.
- Go to the target user → Add permissions → Attach policies directly → select this policy.

SCREENSHOT: IAM policy JSON editor with region-restricted EC2 policy

The screenshot shows the AWS IAM console interface for creating a new policy. The breadcrumb navigation is IAM > Policies > Create policy. The left sidebar shows the 'Specify permissions' step selected. The main area is titled 'Specify permissions' with a sub-header 'Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.' The 'Policy editor' is open in the 'JSON' tab, showing the following JSON:

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Sid": "AllowEC2InSpecificRegions",
6        "Effect": "Allow",
7        "Action": "ec2:*",
8        "Resource": "*",
9        "Condition": {
10         "StringNotEquals": {
11           "aws:RequestedRegion": [
12             "us-east-1",
13             "us-west-2"
14           ]
15         }
16       }
17     ]
18   }
19 }

```

On the right, the 'Edit statement' section is visible, showing a 'Select a statement' button and an 'Add new statement' button.

AWS CLI equivalent:

```

# Create the custom policy
aws iam create-policy \
  --policy-name EC2-Region-Restricted-Policy \
  --policy-document file://ec2-region-restricted-policy.json

# Attach it to the specific user
aws iam attach-user-policy \

```



```
--user-name USER_NAME \  
--policy-arn arn:aws:iam::ACCOUNT_ID:policy/EC2-Region-Restricted-Policy
```

6. Cross-Account S3 Access (Account A User Accessing a Bucket in Account B)

This task grants a user in Account A access to an S3 bucket owned by Account B. It requires changes in both accounts: a bucket policy in Account B, and an IAM policy in Account A.

6.1 In Account B — Add a Bucket Policy

STEP 11 Attach the Bucket Policy

Replace ACCOUNT_A_ID and your-bucket-name with actual values:

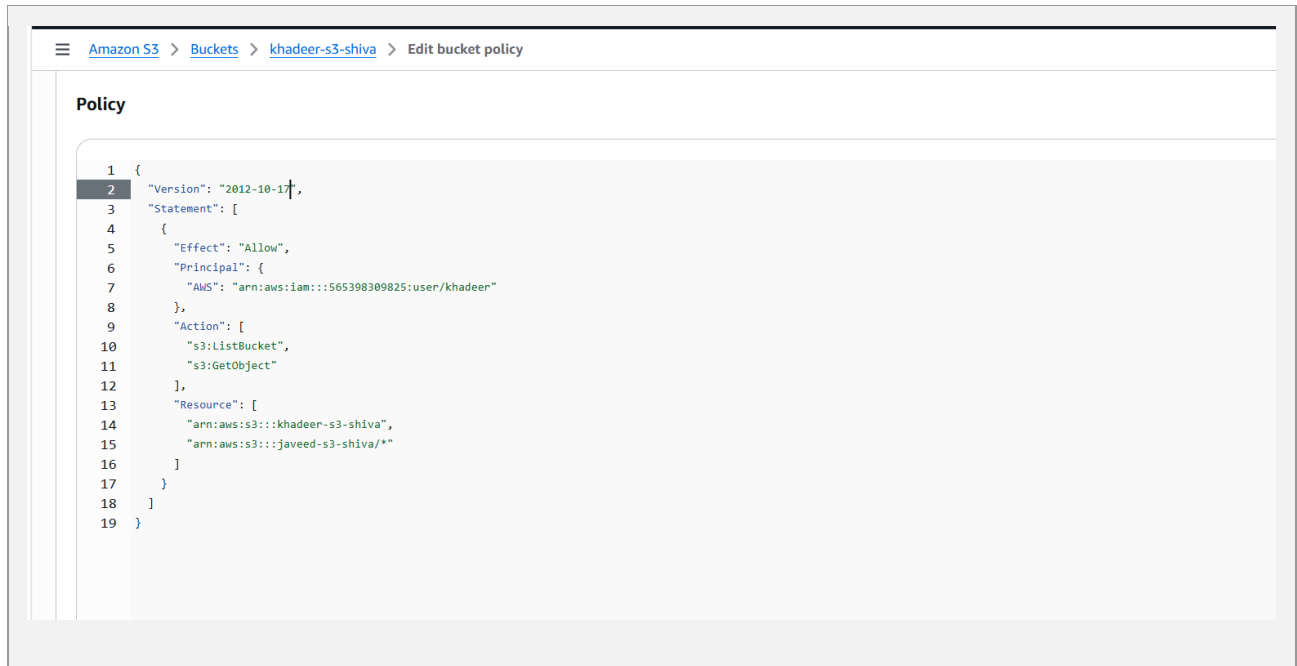
```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "AllowAccountAAccess",  
      "Effect": "Allow",  
      "Principal": { "AWS": "arn:aws:iam::565398309825:user/khadeer" },  
      "Action": ["s3:GetObject", "s3:PutObject", "s3:ListBucket"],  
      "Resource": [  
        "arn:aws:s3:::khadeer-s3-shiva",  
        "arn:aws:s3:::khadeer-s3-shiva/*"  
      ]  
    }  
  ]  
}
```

Apply via CLI (run in Account B):

```
aws s3api put-bucket-policy \  
  --bucket your-bucket-name \  
  --policy file://bucket-policy.json
```



SCREENSHOT: S3 bucket → Permissions → Bucket policy editor (Account B)



6.2 In Account A — Attach an IAM Policy to the User

STEP 12 Create and Attach the Cross-Account Policy

Save as cross-account-s3-access.json:

Apply via CLI (run in Account A):

```
aws iam create-policy \
  --policy-name CrossAccountS3AccessPolicy \
  --policy-document file://cross-account-s3-access.json

aws iam attach-user-policy \
  --user-name USER_NAME_IN_ACCOUNT_A \
  --policy-arn arn:aws:iam::ACCOUNT_A_ID:policy/CrossAccountS3AccessPolicy
```

STEP 13 Verify Access

The Account A user should log in (or use the CLI with their credentials) and test:

```
aws s3 ls s3://khadeer-s3-shiva
```

```
DELL@KHADEER MINGW64 ~  
$ aws --version  
aws-cli/1.45.48 Python/3.14.4 Windows/11 botocore/1.43.48  
  
DELL@KHADEER MINGW64 ~  
$ aws configure  
AWS Access Key ID [*****B3HQ]: AKIAYHJC5RPA7PBRAQXZ  
AWS Secret Access Key [*****9gDw]: 8rvCHFZZTBLSPNp56urxGOJoC5DJpNF1t4  
73Zcn1  
Default region name [us-east-1]:  
Default output format [json]: json  
  
DELL@KHADEER MINGW64 ~  
$ aws sts get-caller-identity  
{  
  "UserId": "AIDAYHJC5RPAQ664LD5QK",  
  "Account": "565398309825",  
  "Arn": "arn:aws:iam::565398309825:user/khadeer"  
}  
  
DELL@KHADEER MINGW64 ~  
$ aws s3 ls  
2026-07-20 15:38:47 khadeers3  
2026-07-16 19:58:08 my-vpc-flowlogs-0329-565398309825-us-east-1-an  
  
DELL@KHADEER MINGW64 ~  
$ aws s3 ls s3://khadeer-s3-shiva  
2026-07-22 17:12:50      502559 Screenshot 2026-07-20 003326.png
```